Concord: The Real Product

1. Why Concord exists

Concord started as Concord Lite, a hackathon prototype that proved a simple idea: a multi-agent workflow can look successful while violating the contracts that make it trustworthy.

The prototype showed one broken AG2 workflow, one diagnostic pipeline, and one final report. That was enough for a hackathon. It is not enough for a product.

The real Concord is the correctness layer for multi-agent workflows.

Concord sits above agent frameworks. It consumes traces, declared workflow contracts, tool events, state snapshots, and approval records. Then it answers five questions:

1. What contract was violated?
2. Which workflow primitive caused or allowed the violation?
3. What repair patch should be applied?
4. What regression test prevents this from recurring?
5. Has this pattern happened before?

2. Concord Lite vs Concord

Dimension	Concord Lite	Concord v1.0	Concord Platform
Goal	Win hackathon with a sharp vertical slice	Become a real AG2-first developer product	Become framework-agnostic correctness control plane
Workflow source	Curated Zone A Literature Review Assistant	User-registered AG2 workflows	AG2, LangGraph, CrewAI, OpenAI Agents SDK, Google ADK, A2A, MCP
Trace source	Fixture or local run	Native AG2 trace ingestion and POST /api/runs	Framework adapters and OpenTelemetry/OpenInference bridges
Storage	In-memory or fixture data	SQLite/Postgres plus run history	Postgres plus FalkorDB graph and memory layer
Contracts	3 deterministic checks + declared future checks	Evidence, Tool, Routing, Approval, Schema contracts	Custom DSL, contract packs, inferred contracts
Repair	One primary patch	One patch per violation	Repair-test-iterate loops and patch history
Test	Generated demo test	Daytona-validated regression tests	AgentCI test suite and CI/CD integration
Dashboard	Frontend-first demo	API-backed live dashboard	Multi-tenant ops console
Trust model	Honest demo fallback	Deterministic checks own verdicts	Auditable, tenant-isolated, policy-governed platform

3. The product boundary

Concord is not a generic tracing product.

AG2 and other frameworks already provide tracing and observability primitives. Concord uses those as input.

Concord is not AG2 Failure Attribution.

AG2 Failure Attribution asks who failed and when. Concord asks what contract broke, which primitive to repair, and what test prevents recurrence.

Concord is not a no-code workflow builder.

It does not try to replace AG2 Studio or framework-specific builders. It verifies and repairs workflows after they are designed.

Concord is not an autonomous patch applier.

Every repair is a recommendation by default. Human approval remains required before applying or exporting repairs.

4. The long-term platform map

The early brainstorming separated Concord into named modules:

Earlier idea	Role inside the real Concord
Janus	Failure context and trace-to-repair support
Consul	Future A2A remote-agent trust and admission
HandoffLint	Routing contract verification
StateGuard	ContextVariables and shared-state contract verification
ToolTruth	Tool-event and tool-claim auditing
GuardrailGen	Failure-to-guardrail repair generation
AgentCI	Regression test generation and CI integration

The key strategic correction is that these should not be seven separate products.

They are internal engines inside one loop:

```
Trace -> Contract Violation -> Affected Primitive -> Repair Patch -> Regression Test -> Memory
```

5. Why the current Concord wedge is stronger

The broad platform claim is too wide for early execution.

A vague pitch says:

```
We solve tracing, trust, routing, state, tools, guardrails, and CI.
```

A sharper Concord pitch says:

```
This workflow violated its contract. Concord found the violated contract, identified the broken primitive, g
```

That is easier to build, easier to demo, and easier to sell.

6. The real product loop

Step 1: Register workflow

The developer registers an AG2 workflow:

- agents
- topology
- tools
- contract rules
- approval rules
- optional source links

Step 2: Submit run

The developer submits:

- native AG2 trace
- final output
- tool events
- ContextVariables snapshots
- approval records

Step 3: Normalize trace

Concord converts raw trace data into a canonical RunTrace:

- agent turns
- handoffs
- tool calls
- code execution
- human input
- speaker selection
- state updates

Step 4: Enforce contracts

Deterministic contract engine checks:

- evidence contract
- tool contract
- routing contract
- approval contract
- schema contract

Step 5: Generate per-violation repair

Each violation gets:

- affected primitive
- before/after
- patch summary
- patch code or config
- expected impact
- approval requirement

Step 6: Generate regression test

Each violation becomes a test:

- should fail on broken trace
- should pass after repair
- can run in Daytona or local fallback

Step 7: Store memory

Concord stores:

- recurring violation patterns
- contract history
- repair success rate
- workflow topology
- agent/tool risk profile

Step 8: Improve future runs

The product gradually becomes a correctness memory for agent workflows.

7. First user

The first user is an AG2 developer moving from demo agents to real workflows.

They have:

- a GroupChat or Swarm-like workflow
- tools

- handoffs
- shared state
- human approval points
- repeated subtle failures

They need:

- deterministic checks
- evidence-based violations
- clear repairs
- regression tests
- repeatability

8. Long-term product vision

Concord becomes the CI/CD and safety control plane for multi-agent workflows.

Long term, it should support:

- AG2 native tracing ingestion
- workflow contracts
- custom DSL
- repair patch generation
- Daytona sandbox validation
- live dashboard streaming
- multi-tenant run history
- FalkorDB topology and recurrence memory
- A2A remote-agent trust
- framework adapters
- CI integration
- contract packs

The full vision:

```
Agent frameworks run workflows.
Concord verifies, repairs, and remembers them.
```

9. The sentence everyone should use

Concord is the contract-to-repair layer for multi-agent workflows.

It turns failed runs into violated contracts, repair patches, and regression tests. # Concord PRD

1. Product summary

Concord is an AG2-first contract-to-repair platform for multi-agent workflows.

It watches a multi-agent workflow run, checks the run against explicit workflow contracts, detects contract violations, maps each violation to the affected workflow primitive, recommends a repair patch, validates the repair through a regression test, and stores the result for future prevention.

Concord Lite proved the concept in a hackathon. Concord v1.0 turns it into a real developer tool.

2. Product thesis

Agent frameworks make it easier to build multi-agent workflows. They do not fully solve workflow correctness.

Developers still need to know:

- Did the workflow follow its declared contract?
- Did the right agents run in the right order?
- Did agents use the tools they claimed to use?
- Did the final answer have required evidence?
- Did side effects require approval?

- What exact primitive should be changed?
- What test prevents this from happening again?

Concord exists to answer those questions.

3. What Concord is not

Concord is not a generic chatbot.

Concord is not a generic observability dashboard.

Concord is not a replacement for AG2 Failure Attribution.

Concord is not a no-code workflow builder.

Concord is not trying to support every multi-agent framework in v1.0.

Concord is AG2-first. The long-term architecture can support other frameworks through adapters, but v1.0 should win by being excellent for AG2 developers.

4. Target users

Primary user: AG2 developer

A developer building AG2 workflows with GroupChat, handoffs, tools, guardrails, and human approval.

Pain:

- Workflows fail subtly.
- Logs are long.
- Final outputs can look correct while violating evidence or routing rules.
- Fixes are unclear.
- Regression tests are rare.

Secondary user: AI platform engineer

A platform engineer responsible for making internal agent systems safe enough for production.

Pain:

- Multiple teams build agents differently.
- There is no central contract enforcement layer.
- They need auditability, cost visibility, and approval controls.

Tertiary user: research automation team

A team using multi-agent workflows for literature review, lab analysis, or experiment planning.

Pain:

- Research agents can produce confident unsupported claims.
- Evidence provenance matters.
- They need repeatability and review trails.

5. Jobs to be done

JTBD 1: Register workflow contracts

When I build an AG2 workflow, I want to declare the workflow's correctness rules so Concord can detect when the workflow violates them.

JTBD 2: Submit a run

When a workflow runs, I want to send Concord the trace so Concord can inspect what happened.

JTBD 3: Detect violations

When a run fails or produces suspicious output, I want Concord to tell me which contracts were violated.

JTBD 4: Get a repair

When a contract is violated, I want Concord to map the violation to the concrete primitive I should change.

JTBD 5: Generate a regression test

When Concord finds a violation, I want it to generate a test that prevents the same failure from returning.

JTBD 6: Watch runs live

When a workflow is in progress, I want the dashboard to show agent turns, tool calls, state updates, violations, and repairs in real time.

JTBD 7: Track recurring failures

When the same workflow violates contracts repeatedly, I want Concord to surface the pattern and recommend a systemic fix.

6. v1.0 scope

P0: must ship

P0.1 Workflow registration Endpoint:

`POST /api/workflows`

User registers:

- workflow name
- declared topology
- agents
- tools
- contracts
- owner/team
- optional source link

Acceptance criteria:

- Workflow persists across server restart.
- Workflow has unique ID.
- Workflow can be fetched and listed.
- Contract schema validates on create.

P0.2 Run submission Endpoint:

`POST /api/runs`

User submits:

- workflow ID
- raw trace
- final output
- ContextVariables snapshot
- tool events
- optional task spec

Acceptance criteria:

- Run persists.
- Run is associated with workflow.
- Run status transitions through queued, analyzing, completed, failed.

- Run can be fetched.

P0.3 Deterministic contract engine Supported contracts:

- Evidence contract
- Tool contract
- Routing contract
- Approval contract
- Schema contract

Acceptance criteria:

- Contract verdicts are deterministic.
- LLM is not required for the verdict.
- LLM may explain the verdict.
- Routing and Schema contracts are enforced, closing the Lite gap.

P0.4 Per-violation repair patches Each violation gets its own repair patch.

Repair patch contains:

- affected primitive
- patch summary
- before/after
- suggested code or config
- confidence
- expected impact
- human approval requirement

Acceptance criteria:

- No single primary-patch limitation.
- Dashboard shows real per-violation patches.

P0.5 Regression test generation Each violation gets a regression test.

Test contains:

- test name
- assertions
- code
- expected failure on broken trace
- expected pass after patch or simulated patch

Acceptance criteria:

- Test can run locally or through Daytona.
- If Daytona is unavailable, simulated fallback is explicit.

P0.6 Daytona validation Use Daytona for sandboxed validation.

Acceptance criteria:

- At least one generated regression test runs in Daytona.
- Report includes pass/fail, stdout, stderr, and sandbox metadata.
- Sandbox cleanup is reliable.

P0.7 API-backed dashboard Dashboard must not stay fixture-only.

Screens:

- overview
- workflow topology
- agent trace

- violations
- repair patch
- regression test
- final report

Acceptance criteria:

- Dashboard can load a real run by run ID.
- Fixture mode remains for demos.
- UI clearly distinguishes live vs fixture data.

P0.8 Persistent storage Use relational storage for product data.

Local dev:

- SQLite acceptable.

Production target:

- Postgres.

Acceptance criteria:

- Workflows, runs, contracts, violations, repairs, tests, and tool events persist.

P1: should ship shortly after v1.0

- Native AG2 tracing ingestion.
- Concord SDK.
- Live dashboard streaming through SSE/WebSocket or AG-UI adapter.
- FalkorDB workflow graph.
- Recurring violation memory.
- Repair-test-iterate loop.
- Slack/Discord/email notifications for approvals.
- Cost dashboard.
- Contract DSL in YAML.

P2: later

- DocAgent-assisted contract extraction from design docs.
- WebSurfer-assisted external verification.
- AG2 Studio import/export or contract visualization.
- A2A remote-agent trust and agent passports.
- LangGraph adapter.
- Contract pack marketplace.
- Advanced repair ranking using ReasoningAgent.

7. Non-goals

Not v1.0:

- Supporting every multi-agent framework.
- No-code workflow builder.
- Full LangSmith/AgentOps replacement.
- Fully automatic patch application.
- Enterprise SSO beyond a basic design.
- Voice approval through Twilio.

Avoid:

- Claims that Concord replaces AG2 Failure Attribution.
- Claims that agents are always right.
- Claims that repairs are safe without human approval.
- Building broad observability instead of contract verification.

8. Success metrics

Product metrics:

- First workflow registered and analyzed in under 10 minutes.
- 90% of runs produce a complete report.
- 95% deterministic contract verdict reproducibility.
- 80% of generated regression tests syntactically valid.
- 50% of patches copied, approved, or used by early users.

Developer metrics:

- SDK integration under 10 lines.
- Local setup under 15 minutes.
- First run visible in dashboard under 2 minutes after submission.

Reliability metrics:

- No tenant isolation failures in tests.
- No unapproved side-effect patch application.
- Daytona sandbox cleanup success above 99%.
- Every violation links to trace evidence.

9. v1.0 launch criteria

Concord v1.0 is ready when a developer can:

1. Create an API key.
2. Register an AG2 workflow.
3. Submit a trace or run task.
4. See live run status.
5. View contract violations.
6. See one repair per violation.
7. Run at least one regression test in Daytona.
8. Export a report.
9. Return later and see persisted history.

10. Open questions

1. Should contract DSL be YAML-only in v1.0, or should Python remain first-class?
2. Should Concord store raw messages by default, or redact sensitive content?
3. How much source-code access is required for diff-grade repair patches?
4. What is the minimum viable tenant/auth model for pilots?
5. Should Daytona be required, or optional with local fallback?
6. Should FalkorDB be required for v1.0 or P1?
7. What contract packs ship by default? # Concord Architecture

1. Architecture summary

Concord v1.0 is an AG2-first contract-to-repair system.

It has five layers:

1. Client layer
2. API layer
3. Worker / diagnostic engine
4. Persistence layer
5. Sandbox / integration layer

The key product path:

```
workflow registration
  -> run submission
```

```
  -> trace normalization
  -> deterministic contract checks
  -> diagnostic agent explanation
  -> repair patch generation
  -> regression test generation
  -> Daytona validation
  -> report persistence
  -> live dashboard update
```

2. Component overview

2.1 Frontend

Responsibilities:

- workflow list
- workflow detail
- run dashboard
- live trace view
- violation detail
- repair patch diff
- regression test result
- final report
- cost view
- approval actions

Recommended migration:

```
frontend/
  src/
    app/
    components/
    pages/
    lib/api.ts
    lib/types.ts
```

Keep `public/` fixture mode until the new app is stable.

2.2 API server

Responsibilities:

- auth
- tenant isolation
- workflow registration
- run submission
- report retrieval
- live status streaming
- dashboard data API

Suggested stack:

- FastAPI
- SQLAlchemy or SQLModel
- Pydantic models
- WebSocket or SSE
- background worker queue

Suggested paths:

```
api/
  index.py
  routes/
```

```
    workflows.py
    runs.py
    reports.py
    auth.py
    streaming.py
  schemas.py
  db.py
  models.py
  auth.py
  tenancy.py
  costs.py
```

2.3 Worker / diagnostic engine

Responsibilities:

- normalize traces
- run contract checks
- run diagnostic GroupChat if needed
- generate repairs
- generate regression tests
- call Daytona
- call Tavily
- persist outputs

Suggested paths:

```
worker/
  main.py
  jobs.py
  queue.py

zone_b/
  orchestrator.py
  group_chat.py
  contracts/
    types.py
    registry.py
    checks.py
    dsl.py
  agents/
    trace_collector.py
    contract_checker.py
    attribution.py
    repair.py
    regression_test.py
    reporter.py
  sandbox/
    daytona_pool.py
    runner.py
  memory/
    violation_memory.py
```

2.4 SDK

Responsibilities:

- instrument AG2 workflow
- emit traces
- submit runs

- optionally register workflow
- local dev helper

Suggested paths:

```
sdk/
  concord_sdk/
    __init__.py
    client.py
    instrumentation.py
    schemas.py
```

2.5 Persistence layer

Use two stores:

1. Relational DB for transactional product data.
2. FalkorDB graph for workflow topology and violation pattern memory.

P0 can start with SQLite for local dev and a Postgres-compatible schema.

P1 adds FalkorDB.

3. Data flow

3.1 Register workflow

```
client
  -> POST /api/workflows
  -> validate workflow schema
  -> persist workflow
  -> persist contracts
  -> optionally persist topology in FalkorDB
  -> return workflow_id
```

3.2 Submit run trace

```
client or SDK
  -> POST /api/runs
  -> create run row
  -> enqueue analysis job
  -> return run_id
```

3.3 Analyze run

```
worker
  -> load workflow contract
  -> normalize trace
  -> run deterministic contract checks
  -> create violations
  -> run diagnostic agents for explanation and repair narrative
  -> generate per-violation repair patches
  -> generate tests
  -> run or simulate tests in Daytona
  -> persist report
  -> emit dashboard events
```

3.4 Dashboard live update

```
frontend subscribes to /api/runs/{id}/events
  -> RUN_STARTED
  -> TRACE_NORMALIZED
```

```
  -> CONTRACT_CHECKED
  -> VIOLATION_FOUND
  -> REPAIR_GENERATED
  -> TEST_RUNNING
  -> TEST_PASSED or TEST_FAILED
  -> REPORT_READY
```

4. Core schemas

Workflow

```
{
  "workflow_id": "wf_123",
  "tenant_id": "tenant_abc",
  "name": "Literature Review Assistant",
  "framework": "ag2",
  "topology": {
    "agents": [
      {"name": "ResearcherAgent", "type": "ConversableAgent"},
      {"name": "CriticAgent", "type": "ConversableAgent"},
      {"name": "VerifierAgent", "type": "ConversableAgent"},
      {"name": "ReporterAgent", "type": "ConversableAgent"},
      {"name": "ActionAgent", "type": "ConversableAgent"}
    ],
    "edges": [
      {"from": "ResearcherAgent", "to": "CriticAgent"},
      {"from": "CriticAgent", "to": "VerifierAgent"},
      {"from": "VerifierAgent", "to": "ReporterAgent"},
      {"from": "ReporterAgent", "to": "ActionAgent"}
    ]
  },
  "contracts": []
}
```

RunTrace

```
{
  "run_id": "run_123",
  "workflow_id": "wf_123",
  "events": [
    {
      "event_id": "evt_1",
      "step": 1,
      "timestamp": "2026-05-03T12:00:00Z",
      "agent": "ResearcherAgent",
      "type": "agent_turn",
      "content": "...",
      "tool_call_id": null,
      "context_delta": {},
      "handoff_to": "CriticAgent"
    }
  ],
  "final_output": {}
}
```

Contract

```
{
  "contract_id": "contract_123",
```

```
  "code": "C-EVD",
  "type": "evidence",
  "name": "Reporter requires verified sources",
  "severity": "high",
  "rule": {
    "agent": "ReporterAgent",
    "requires": {
      "verified_sources_count": {"gt": 0}
    }
  }
}
```


Violation


```
{
  "violation_id": "v_123",
  "run_id": "run_123",
  "contract_id": "contract_123",
  "contract_code": "C-EVD",
  "severity": "high",
  "expected": "ReporterAgent requires verified_sources_count > 0",
  "observed": "verified_sources_count = 0",
  "evidence": [
    {
      "event_id": "evt_6",
      "step": 6,
      "agent": "ReporterAgent",
      "field": "verified_sources_count",
      "value": 0
    }
  ],
  "failed_agent": "ReporterAgent",
  "failed_step": 6,
  "affected_primitive": "Guardrail"
}
```


RepairPatch


```
{
  "repair_id": "repair_123",
  "violation_id": "v_123",
  "affected_primitive": "Guardrail",
  "title": "Block Reporter without verified sources",
  "summary": "Add a guardrail requiring verified_sources_count > 0 before ReporterAgent can produce the fina
  "before": "ReporterAgent can run with verified_sources_count = 0",
  "after": "ReporterAgent is blocked or routed back to ResearcherAgent",
  "patch_code": "...",
  "requires_approval": true
}
```


RegressionTest


```
{
  "test_id": "test_123",
  "violation_id": "v_123",
  "name": "test_reporter_requires_verified_sources",
  "assertions": [
    "ReporterAgent must not produce final output when verified_sources_count == 0"
  ],
```

```
  "code": "...",
  "status": "passed",
  "runner": "daytona",
  "stdout": "...",
  "stderr": ""
}
```

5. Relational schema

Use SQLite locally, Postgres in production.

Tables:

```
tenants
api_keys
workflows
contracts
runs
trace_events
violations
repair_patches
regression_tests
tool_events
cost_records
approval_events
```

Every tenant-owned row must contain `tenant_id`.

Critical indexes

```
workflows(tenant_id)
runs(tenant_id, workflow_id, created_at)
violations(tenant_id, run_id, contract_code)
repair_patches(tenant_id, violation_id)
regression_tests(tenant_id, violation_id)
trace_events(tenant_id, run_id, step)
tool_events(tenant_id, run_id, tool_name)
```

6. FalkorDB graph schema

Use FalkorDB for topology, contract relationships, and recurrence memory.

Nodes

```
(:Tenant {id, name})
(:Workflow {id, name, framework})
(:Agent {id, name, type})
(:Tool {id, name, type})
(:Contract {id, code, type, severity})
(:Run {id, status, created_at})
(:TraceEvent {id, step, type, timestamp})
(:Violation {id, code, severity})
(:RepairPatch {id, affected_primitive})
(:RegressionTest {id, status})
(:Document {id, source_type, uri})
(:Pattern {id, fingerprint, count})
```

Relationships

```
(:Tenant)-[:OWNS]->(:Workflow)
(:Workflow)-[:HAS_AGENT]->(:Agent)
(:Workflow)-[:HAS_TOOL]->(:Tool)
(:Workflow)-[:HAS_CONTRACT]->(:Contract)
(:Workflow)-[:HAS_RUN]->(:Run)
(:Run)-[:HAS_EVENT]->(:TraceEvent)
(:Run)-[:HAS_VIOLATION]->(:Violation)
(:Violation)-[:VIOLATES]->(:Contract)
(:Violation)-[:FAILED_AT]->(:TraceEvent)
(:Violation)-[:INVOLVES_AGENT]->(:Agent)
(:Violation)-[:REPAIRED_BY]->(:RepairPatch)
(:RepairPatch)-[:VALIDATED_BY]->(:RegressionTest)
(:Agent)-[:HANDOFF_TO]->(:Agent)
(:Agent)-[:USES_TOOL]->(:Tool)
(:Violation)-[:MATCHES_PATTERN]->(:Pattern)
(:Document)-[:DECLARES_INTENT_FOR]->(:Workflow)
```

Example graph queries

Find recurring violations:

```
MATCH (w:Workflow {id: $workflow_id})-[:HAS_RUN]->(r:Run)-[:HAS_VIOLATION]->(v:Violation)
RETURN v.code, v.affected_primitive, count(*) AS count
ORDER BY count DESC
```

Find unsafe path to ActionAgent:

```
MATCH p = (a:Agent {name: "ReporterAgent"})-[:HANDOFF_TO*1..3]->(b:Agent {name: "ActionAgent"})
WHERE NONE(n IN nodes(p) WHERE n.name = "HumanGate")
RETURN p
```

7. API contract

POST /api/workflows

Request:

```
{
  "name": "Literature Review Assistant",
  "framework": "ag2",
  "topology": {},
  "contracts": []
}
```

Response:

```
{
  "workflow_id": "wf_123",
  "status": "created"
}
```

POST /api/runs

Request:

```
{
  "workflow_id": "wf_123",
  "trace": {},
  "final_output": {},
  "mode": "analyze_trace"
}
```

Response:

```
{
  "run_id": "run_123",
  "status": "queued"
}
```

GET /api/runs/{run_id}

Response includes:

- run metadata
- normalized trace
- violations
- repairs
- regression tests
- report
- cost data

GET /api/runs/{run_id}/events

Use WebSocket or SSE.

Event examples:

```
{"type": "RUN_STARTED", "run_id": "run_123"}
{"type": "TRACE_NORMALIZED", "event_count": 42}
{"type": "VIOLATION_FOUND", "violation_id": "v_123", "code": "C-EVD"}
{"type": "REPAIR_GENERATED", "repair_id": "repair_123"}
{"type": "TEST_PASSED", "test_id": "test_123"}
{"type": "REPORT_READY", "run_id": "run_123"}
```

8. AG2 pattern usage

Product area	AG2 feature	Use
Diagnostic orchestration	GroupChat / GroupChatManager	Multi-agent diagnostic pipeline
Stable stage flow	RoundRobin or deterministic sequential orchestration	Reliable baseline
Contract routing repairs	Handoffs / OnContextCondition	Generate AG2-native routing patches
Shared state checks	ContextVariables	Validate workflow state requirements
Safety checks	RegexGuardrail and output guardrails	Catch missing citations, schema issues, unsafe output
Human approval	UserProxyAgent	Gate repair approval
Tool evidence	TavilySearchTool	External evidence verification
Test validation	DaytonaCodeExecutor	Run tests in sandbox
Live dashboard	AG-UI or custom SSE/WebSocket	Stream agent and tool events
Native traces	autogen.opentelemetry	Replace hand-rolled trace capture
Graph memory	FalkorDB GraphRAG	Workflow topology and recurrence memory

9. Deployment topology

Local development

```
frontend dev server
FastAPI API server
worker process
SQLite
optional FalkorDB Docker container
optional Daytona cloud sandbox
```

Pilot deployment

```
Vercel or frontend host
API container
worker container
Postgres
FalkorDB
Redis queue
Daytona cloud
Tavily API
Gemini/OpenRouter
```

Production target

```
CDN / frontend
API service
worker service
Postgres primary
FalkorDB graph store
Redis / queue
object storage for trace archives
secrets manager
observability backend
Daytona sandbox pool
```

10. Security model

Authentication

P0:

- API key per tenant.

P1:

- dashboard login.
- SSO for pilots if required.

Authorization

Every request resolves:

```
api_key -> tenant_id -> allowed workflow/run access
```

No cross-tenant reads.

Secrets

Never store raw API keys.

Store:

- hash of Concord API key
- encrypted external provider keys if users supply them
- environment-managed Daytona/Tavily/Gemini keys for internal runs

Trace privacy

Traces can contain sensitive prompts, documents, or tool outputs.

Policies:

- raw trace storage optional per tenant
- redaction hooks before persistence

- sensitive keys never written to trace
- UI labels fixture vs live data

Sandbox safety

Daytona is preferred for executing generated regression tests.

Rules:

- no local execution of untrusted generated code in production
- timeout on every execution
- resource limits
- cleanup after execution
- captured stdout/stderr only
- no secrets in sandbox unless explicitly required

Repair safety

Repairs are recommendations by default.

No automatic patch application in v1.0 without human approval.

11. Reliability model

Every LLM call must have a deterministic fallback.

Every contract verdict must be reproducible.

Every repair must name an affected primitive.

Every generated test must include assertions.

Every report must cite trace evidence.

If a tool fails, report status should say:

```
tool_status = failed
verdict_source = deterministic_check
tool_evidence = unavailable
```

Do not silently downgrade failed tools to success. # Concord AG2 Leverage Plan

1. Principle

Concord should use AG2 deeply but not recklessly.

The goal is not to name-drop every AG2 feature. The goal is to use AG2 where it makes the product more correct, inspectable, reliable, or faster to build.

Rule:

```
Use AG2 primitives when they directly support contract checking, trace ingestion, repair generation, test va
```

2. P0 AG2 usage

AG2 feature	Module path / import shape	Concord use	Decision
GroupChat	`from autogen import GroupChat, GroupChatManager` or current installed equivalent	Diagnostic multi-agent pipeline	Use
GroupChatManager	`from autogen import GroupChatManager`	Orchestrates diagnostic agents	Use

AG2 feature	Module path / import shape	Concord use	Decision
ConversableAgent	`from autogen import ConversableAgent`	Base class for diagnostic agents	Use
UserProxyAgent	`from autogen import UserProxyAgent`	Human approval gate	Use
ContextVariables	`from autogen.agentchat.group import ContextVariables`	Store workflow state, contract state, repair state	Use
OnContextCondition	`from autogen.agentchat.group import OnContextCondition`	Deterministic routing based on context variables	Use
Handoffs	`from autogen.agentchat.group.handoffs import Handoffs`	Production handoff repair and diffs	Use
RegexGuardrail	Verify exact installed import path	Citation/schema safety checks	Use
TavilySearchTool	`from autogen.tools.experimental import TavilySearchTool`	External evidence verification	Use
DaytonaCodeExecutor	`from autogen.coding import DaytonaCodeExecutor`	Run generated regression tests in sandbox	Use
LLMConfig	`from autogen import LLMConfig`	Gemini/OpenRouter/provider routing config	Use
AG2 OpenTelemetry	`from autogen.opentelemetry import instrument_agent, instrument_llm_wrapper, instrument_pattern`	Native trace ingestion	P1, but design now

3. P1 AG2 usage

AG2 feature	Module path / import shape	Concord use	Decision
AG-UI	`from autogen.ag_ui import AGUIStream`	Live dashboard streaming	Use if it fits dashboard event model
FalkorDB GraphRAG	`autogen.agentchat.contrib.graph_rag.falkor_graph_query_engine.FalkorGraphQueryEngine` and `autogen.agentchat.contrib.graph_rag.falkor_graph_rag_capability.FalkorGraphRagCapability`	Workflow topology, graph and recurrence memory	Use
Mem0	`from mem0 import Memory`	Long-term semantic memory of recurring violation patterns	Use if graph alone is not enough
Nested chats	`ConversableAgent.register_nested_chats(...)`	Isolate Tests (>.R) repair loop	Use after single-pass loop works
Custom speaker selection	`GroupChat(..., speaker_selection_method=callable)`	Skip unnecessary validation stages	Use after baseline stability
DocAgent	`from autogen.agents.experimental import DocAgent`	Ingest design docs and draft contracts	Use after core contracts work

AG2 feature	Module path / import shape	Concord use	Decision
ReasoningAgent	`from autogen.agents.experimental import ReasoningAgent`	Complex multi-violation root-cause ranking	Use only for hard cases

4. P2 / optional AG2 usage

AG2 feature	Module path / import shape	Concord use	Decision
WebSurferAgent	`from autogen.agents.experimental import WebSurferAgent`	Browse external docs when contract references live pages	Optional
CaptainAgent	`from autogen.agentchat.contrib.captain_agent import CaptainAgent`	Decide whether to invoke data agent or assemble specialists	Not v1 core
AG2 Studio	Verify current integration path via official docs/repo	Visual import/export or contract visualization	Optional
A2A	`autogen.a2a.*`	Remote agent trust and Agent Passport later	Not v1 core
Twilio / RealtimeAgent	Current docs should be verified; avoid if deprecated	Voice approval	Avoid for v1
AG2 GraphQL	Verify via docs before planning	Dashboard query layer	Do not plan until verified

5. Code stubs

5.1 Diagnostic GroupChat

```
from autogen import ConversableAgent, GroupChat, GroupChatManager, LLMConfig

llm_config = LLMConfig({
    "model": "gemini-2.5-flash",
    "api_key": "...",
    "api_type": "openai",
    "base_url": "https://openrouter.ai/api/v1",
    "temperature": 0.1,
})

trace_collector = ConversableAgent(
    name="TraceCollectorAgent",
    system_message="Normalize the run trace. Do not invent events.",
    llm_config=llm_config,
    human_input_mode="NEVER",
)

contract_checker = ConversableAgent(
    name="ContractCheckerAgent",
    system_message="Explain deterministic contract violations. Do not decide verdicts.",
    llm_config=llm_config,
    human_input_mode="NEVER",
)

repair_agent = ConversableAgent(
```

```
    name="RepairAgent",
    system_message="Map violations to AG2-native repair patches.",
    llm_config=llm_config,
    human_input_mode="NEVER",
)

group_chat = GroupChat(
    agents=[trace_collector, contract_checker, repair_agent],
    messages=[],
    max_round=8,
    speaker_selection_method="round_robin",
)

manager = GroupChatManager(groupchat=group_chat, llm_config=llm_config)
```

Use RoundRobin for deterministic v1 baseline. Add custom speaker selection later.

5.2 Handoffs repair pattern

```
from autogen.agentchat.group.handoffs import Handoffs
from autogen.agentchat.group import OnContextCondition

# Pseudocode. Adjust target and condition classes to installed AG2 version.
verifier_agent.handoffs = (
    Handoffs()
    .add_context_condition(
        OnContextCondition(
            target=reporter_agent,
            condition=verified_sources_count_gt_zero_condition,
        )
    )
    .set_after_work(researcher_agent)
)
```

Concord should generate this as a suggested repair patch, not auto-apply it without approval.

5.3 Daytona regression test runner

```
from autogen.coding import DaytonaCodeExecutor
from autogen import ConversableAgent


def run_generated_test_in_daytona(test_code: str) -> dict:
    with DaytonaCodeExecutor(timeout=60) as executor:
        code_executor_agent = ConversableAgent(
            name="daytona_test_executor",
            llm_config=False,
            code_execution_config={"executor": executor},
            human_input_mode="NEVER",
        )

        result = code_executor_agent.run(
            message=f"```python\n{test_code}\n```",
            max_turns=1,
        )
        return {
            "status": "completed",
            "summary": getattr(result, "summary", None),
        }
```

5.4 Tavily evidence tool

```
from autogen.tools.experimental import TavilySearchTool

tavily_tool = TavilySearchTool(tavily_api_key=os.environ["TAVILY_API_KEY"])
```

A Tavily call must create a ToolEvent. Concord should flag claims that imply search if no corresponding ToolEvent exists.

5.5 AG2 native tracing adapter

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider

from autogen.opentelemetry import (
    instrument_agent,
    instrument_llm_wrapper,
    instrument_pattern,
)


def instrument_concord_workflow(agents, pattern=None):
    tracer_provider = TracerProvider()
    trace.set_tracer_provider(tracer_provider)

    instrument_llm_wrapper(tracer_provider=tracer_provider)

    for agent in agents:
        instrument_agent(agent, tracer_provider=tracer_provider)

    if pattern is not None:
        instrument_pattern(pattern, tracer_provider=tracer_provider)

    return tracer_provider
```

Build an exporter that maps AG2 spans into Concord TraceEvent.

5.6 FalkorDB GraphRAG

```
from autogen.agentchat.contrib.graph_rag.falkor_graph_query_engine import FalkorGraphQueryEngine
from autogen.agentchat.contrib.graph_rag.falkor_graph_rag_capability import FalkorGraphRagCapability
from autogen import ConversableAgent

query_engine = FalkorGraphQueryEngine(
    name="concord_workflow_graph",
    host=os.environ.get("FALKORDB_HOST", "localhost"),
    port=int(os.environ.get("FALKORDB_PORT", "6379")),
)

graph_agent = ConversableAgent(
    name="ViolationMemoryAgent",
    human_input_mode="NEVER",
)

capability = FalkorGraphRagCapability(query_engine)
capability.add_to_agent(graph_agent)
```

Use direct graph operations for persistence. Use GraphRAG for querying and explanation.

5.7 Mem0 recurrence memory

```python
from mem0 import Memory

memory = Memory()


def remember_violation(tenant_id: str, workflow_id: str, violation_summary: str):
    memory.add(
        messages=[{"role": "system", "content": violation_summary}],
        user_id=f"{tenant_id}:{workflow_id}",
    )


def search_similar_violations(tenant_id: str, workflow_id: str, query: str):
    return memory.search(query, user_id=f"{tenant_id}:{workflow_id}")
```

Use Mem0 for semantic recurrence. Use FalkorDB for exact topology and contract relationships.

5.8 DocAgent contract drafting

```python
from autogen.agents.experimental import DocAgent

doc_agent = DocAgent(
    name="WorkflowIntentDocAgent",
    llm_config=llm_config,
    parsed_docs_path="./data/parsed_docs",
    collection_name="workflow_intent_docs",
)
```

Use case:

```
operator uploads workflow design doc
  -> DocAgent extracts intended behavior
  -> Concord proposes draft contracts
  -> human approves contracts
```

5.9 ReasoningAgent for hard cases

```python
from autogen.agents.experimental import ReasoningAgent

reasoning_agent = ReasoningAgent(
    name="ComplexRootCauseAgent",
    llm_config=llm_config,
    grader_llm_config=llm_config,
    max_depth=3,
    beam_size=3,
)
```

Use only for multi-violation cases where deterministic checks find several possible root causes.

5.10 WebSurferAgent

```python
from autogen.agents.experimental import WebSurferAgent

websurfer = WebSurferAgent(
    name="ExternalDocsAgent",
    web_tool="crawl4ai",
    llm_config=llm_config,
)
```

Use only when a contract references external documentation that Tavily cannot cover cleanly.

5.11 Nested chat for repair-test iteration

```
repair_loop = [
    {
        "recipient": repair_agent,
        "message": "Generate a minimal repair patch for the violation.",
        "max_turns": 1,
        "summary_method": "last_msg",
    },
    {
        "recipient": regression_test_agent,
        "message": "Generate and evaluate a regression test for the repair.",
        "max_turns": 1,
        "summary_method": "last_msg",
    },
]

repair_supervisor.register_nested_chats(
    chat_queue=repair_loop,
    trigger=lambda sender: True,
)
```

Use after single-pass repair generation is stable.

5.12 Custom speaker selection

```
def diagnostic_speaker_selection(last_speaker, groupchat):
    state = extract_state(groupchat.messages)

    if state.get("violation_count") == 0:
        return reporter_agent

    if state.get("violation_count") == 1 and not state.get("needs_complex_attribution"):
        return repair_agent

    if state.get("test_failed"):
        return repair_agent

    return "round_robin"
```

Use only after deterministic baseline is stable.

5.13 CaptainAgent

```
from autogen.agentchat.contrib.captainagent import CaptainAgent

captain = CaptainAgent(
    name="ConcordCaptain",
    llm_config=llm_config,
)
```

Possible use:

- decide whether to run full diagnostics
- assemble specialized agents for unknown contract types
- recommend contract pack

Decision:

- Not v1 core.
- Too much autonomy before contracts are stable.

5.14 Per-provider LLM routing

```python
from autogen import LLMConfig

fast_config = LLMConfig({
    "model": "gemini-2.5-flash",
    "api_key": os.environ["OPENROUTER_API_KEY"],
    "api_type": "openai",
    "base_url": "https://openrouter.ai/api/v1",
    "temperature": 0.1,
})

strong_config = LLMConfig({
    "model": "gemini-2.5-pro",
    "api_key": os.environ["GEMINI_API_KEY"],
    "api_type": "gemini",
    "temperature": 0.1,
})
```

Policy:

```
cheap deterministic explanations -> Gemini Flash
complex repair ranking -> Gemini Pro or stronger model
fallback -> configured secondary provider
```

Track model selection and cost per run.

6. AG2 features to avoid for now

Feature	Why avoid
Random speaker selection	Bad for correctness product
Fully autonomous repair application	Too risky
CaptainAgent in core path	Too much dynamic behavior before contracts are stable
WebSurfer for basic search	Tavily is simpler and more reliable
Twilio RealtimeAgent	Verify current support before building; not needed for v1
AG2 GraphQL	Not verified in official docs during planning
Full AG2 Studio dependency	Useful later, not production core

7. Recommended AG2 sequencing

Now

1. Stabilize deterministic contract engine.
2. Use existing AG2 GroupChat.
3. Use Daytona and Tavily visibly.
4. Keep UserProxyAgent approval.

Next

1. Add native AG2 OpenTelemetry ingestion.
2. Add SDK one-line instrumentation.
3. Add live dashboard streaming.
4. Add persistent workflow/run storage.

Then

1. Add FalkorDB topology and recurrence memory.
2. Add nested repair-test iteration.
3. Add DocAgent contract drafting.

4. Add custom speaker selection.
5. Add Mem0 if semantic recurrence needs it.

Later

1. AG2 Studio import/export.
2. A2A remote-agent trust.
3. LangGraph adapter.
4. Contract marketplace.